

EduStack Enterprise

Complete Technical Project Documentation & Architecture Blueprint

Your Ultimate Computer Science & Engineering Hub

Full Stack Software Architecture | Node.js Express | Python FastAPI | MongoDB | Security | AI Engine

Project Name:	EduStack -- Your Ultimate Computer Science Hub
Developer:	Shubham Kumar CSE Student NIT Patna
Repository:	github.com/ShubhamKumar968/EduStack--Your-Ultimate-Computer-Science-Hub
Primary Runtime:	Node.js 18 LTS + Express.js 4 (REST API Monolith)
Microservice:	Python 3.11 + FastAPI + Uvicorn (AI / ML RAG Service)
Database Layer:	MongoDB Atlas Managed Cluster + Mongoose 7 ODM
Security Engine:	Stateless JWT httpOnly Cookies, bcrypt-12, Helmet, NoSQL Sanitize
Cloud Deployment:	Render.com Web Services + Cloudinary CDN Media Storage

Table of Contents - System Documentation

1.	Executive Summary & System Vision Platform problem statement, core capabilities, engineering goals
2.	System Architecture & Hybrid Infrastructure Node.js monolith + Python FastAPI microservice proxy architecture
3.	Codebase Directory & Module Walkthrough Complete folder structure breakdown of server/, client/, and ml_services/
4.	Stateless Security Engine & Authentication Dual auth (Local + Google OAuth 2.0), JWT httpOnly cookies, bcrypt-12
5.	Database Design & Complete Mongoose Schemas User, OTP TTL, Subject, Resource, Payment, and Notification models
6.	REST API Specification & Route Reference Complete endpoint reference guide with payload structures
7.	Python AI / ML Microservice & RAG Engine FastAPI ASGI server, Google Gemini 1.5/2.0 Flash, LightRAG, PDF parser
8.	Razorpay Payment Gateway & Cryptography 3-step payment lifecycle, HMAC-SHA256 verification, timingSafeEqual
9.	Cloud Deployment & Production Setup Render.com, MongoDB Atlas, Cloudinary zero-disk upload, env setup
10.	Maintenance, Troubleshooting & Operational Playbook Monitoring, error logging, database backups, disaster recovery

Executive Summary & System Vision

Platform problem statement, core capabilities, engineering goals, and domain vision

1.1 The Problem Space

Computer science and engineering education suffers from severe content fragmentation. Previous year exam papers, lecture notes, syllabus blueprints, and competitive programming trackers are scattered across unindexed drives, social media channels, and static websites.

EduStack was engineered as a single unified platform providing academic subjects, organized semester resources, a live competitive DSA problem tracker, an automated AI exam paper tutor, and transactional premium subscriptions.

Core Platform Features

- **Subject Catalog Engine:** Manages 42+ CS subjects categorized by semester with Mongoose schemas, code indexing, and virtual populations.
- **Resource Repository:** Categorized notes, question papers, and lab manuals streamed directly to Cloudinary CDN.
- **DSA Competitive Tracker:** 450+ hand-curated competitive programming problems synced live from a published Google Sheet CSV with an in-memory 5-minute TTL cache and local JSON fallback.
- **AI Exam Tutor & RAG Engine:** Python FastAPI microservice powered by Google Gemini 1.5/2.0 Flash and LightRAG for grounded Q&A and PDF text extraction.
- **Stateless Security Engine:** Dual auth system (Local Email/Password + Google OAuth 2.0) issuing JWTs stored exclusively in httpOnly, Secure cookies.
- **Transactional Payments:** Razorpay payment gateway integration with server-side order creation and HMAC-SHA256 constant-time signature verification.

System Architecture & Hybrid Infrastructure

Node.js Express monolith + Python FastAPI microservice gateway architecture

2.1 Architectural Overview

EduStack utilizes a deliberate hybrid architecture combining the rapid non-blocking I/O of Node.js for web endpoints with the specialized AI/ML capabilities of Python FastAPI.

Component	Technology Stack	Responsibility	Deployment Unit
Web Monolith	Node.js 18 LTS + Express.js 4	REST APIs, auth, routing, database CRUD, payment logic	Render.com Web Service
AI Microservice	Python 3.11 + FastAPI + Uvicorn	Gemini AI inference, RAG retrieval, PDF text extraction	Render.com Web Service (Internal)
Database Layer	MongoDB Atlas M0 Managed Cluster	Persistent storage for users, subjects, resources, payments	MongoDB Atlas Cloud
Media Storage	Cloudinary Media API	CDN storage for user avatars and PDF resource documents	Cloudinary Global CDN

EduStack Codebase Specification

```
// HTTP Proxy Gateway Pattern in server/routes/aiRoutes.js
// Browser communicates ONLY with Node.js. Node.js proxies AI requests to Python microservice.
const ML_SERVICE_URL = process.env.ML_SERVICE_URL || "http://localhost:8000";

router.use(isAuth);           // 1. Verify user JWT token
router.use(aiRateLimiter);    // 2. Apply rate limit (30 req / 10 min per IP)

router.post("/generate-pyq", async (req, res, next) => {
  try {
    const response = await axios.post(`${ML_SERVICE_URL}/api/rag/generate-pyq`, req.body);
    return res.status(response.status).json(response.data);
  } catch (err) {
    return res.status(503).json({ success: false, message: "AI Microservice Offline." });
  }
});
```

Codebase Directory & Module Walkthrough

Complete file structure breakdown of server/, client/, and ml_services/

3.1 Directory Structure

EduStack Codebase Specification

```
EduStack/
% % % server/
% % % % app.js
% % % % config/
% % % % controllers/
% % % % middlewares/
% % % % models/
% % % % routes/
% % % % services/
% % % % utils/
% % % client/
% % % % assets/
% % % % index.html
% % % ml_services/
% % % % main.py
% % % % requirements.txt
% % % generate-part1.js

# Node.js Express Backend Monolith
# Application entry point & middleware registration
# DB, Passport, and Cloudinary configuration
# Route controllers (auth, subject, payment, etc.)
# isAuth, requireRole, errorHandler, rateLimiter
# Mongoose models (User, Subject, Resource, Payment, OTP)
# Express API routers (/api/auth, /api/subjects, etc.)
# Business logic (mailService, razorpayService)
# asyncHandler, apiResponse, generateToken
# Vanilla JS Single Page Application
# Stylesheets, JS Partial components, images
# Main frontend interface
# Python FastAPI AI Microservice
# FastAPI app & Gemini API endpoints
# Python dependencies (fastapi, google-generativeai)
# Masterclass PDF Generators (Part 1 to Part 4)
```

Stateless Security Engine & Authentication

Dual authentication (Local + Google OAuth 2.0), JWT httpOnly cookies, bcrypt-12

4.1 Stateless Security Pipeline

EduStack implements OWASP-compliant stateless security storing JWTs exclusively in httpOnly, Secure, SameSite cookies to protect tokens against Cross-Site Scripting (XSS) theft.

Security Mechanism	Implementation Detail	Vulnerability Prevented
httpOnly Cookies	Token set via Set-Cookie header with httpOnly:true	Cross-Site Scripting (XSS) token theft
bcrypt 12 Rounds	4096 salt rounds iteration (~300ms hash time)	GPU brute-force & Rainbow table attacks
mongoSanitize	Strips \$ and . characters from req.body/params/query	NoSQL Operator Injection attacks
Helmet Headers	Attaches 15+ HTTP security headers (HSTS, Clickjacking)	Clickjacking, MIME-sniffing, SSL stripping
timingSafeEqual	Constant-time buffer comparison for Razorpay HMAC	Timing side-channel key recovery attacks

EduStack Codebase Specification

```
// JWT Generation & Cookie Attachment in server/utils/generateToken.js
const generateToken = (res, userId) => {
  const token = jwt.sign({ id: userId }, process.env.JWT_SECRET, { expiresIn: "7d" });
  res.cookie("edustack_token", token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    sameSite: process.env.NODE_ENV === "production" ? "none" : "lax",
    maxAge: 7 * 24 * 60 * 60 * 1000
  });
  return token;
};
```

Database Design & Complete Mongoose Schemas

User, OTP TTL, Subject, Resource, Payment, and Notification models

5.1 Database Schema Schematics

MongoDB Atlas manages data using 6 primary Mongoose schemas:

- **User Schema:** Manages credentials, hashed password (select: false), role enum ("user", "student", "contributor", "admin"), verification status, and premium subscription state.
- **OTP Schema:** Managed with MongoDB TTL Index ({ createdAt: 1 }, { expireAfterSeconds: 600 }) for automatic 10-minute code expiration.
- **Subject Schema:** CS subjects indexed by semester and subject code with virtual references to attached resources.
- **Resource Schema:** Study documents, notes, and previous year papers linked to subjects with Cloudinary CDN URLs.
- **Payment Schema:** Tracks Razorpay orders, payment IDs, signature hashes, and payment verification status.
- **Notification Schema:** System-wide alerts and user notifications.

EduStack Codebase Specification

```
// server/models/user.js - User Schema Blueprint
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, required: true, select: false },
  role: { type: String, enum: ["user", "student", "contributor", "admin"], default: "user" },
  isVerified: { type: Boolean, default: false },
  isPremium: { type: Boolean, default: false },
  avatar: { type: String, default: "" }
}, { timestamps: true });
```

REST API Specification & Route Reference

Complete endpoint reference guide with payload structures and HTTP status codes

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	None	Register new user account with email normalization and avatar upload
POST	/api/auth/verify-otp	None	Verify 6-digit email OTP and activate user account
POST	/api/auth/login	None	Authenticate user and set httpOnly JWT cookie
GET	/api/auth/me	isAuth	Get current authenticated user profile
GET	/api/subjects	isAuth	List all computer science subjects categorized by semester
POST	/api/subjects	isAuth + Admin	Create new subject entry (Admin only)
POST	/api/ai/generate-pyq	isAuth + RateLimit	Generate AI exam paper answers via Python FastAPI microservice
POST	/api/payments/create-order	isAuth	Initialize Razorpay payment order for Premium upgrade
POST	/api/payments/verify	isAuth	Verify Razorpay HMAC signature and activate Premium status

Python AI / ML Microservice & RAG Engine

FastAPI ASGI server, Google Gemini 1.5/2.0 Flash, LightRAG, PDF text extraction

The AI engine runs in an isolated Python FastAPI container utilizing Google Gemini API and LightRAG for context-grounded educational tutoring.

EduStack Codebase Specification

```
# ml_services/main.py - Python FastAPI AI Microservice
from fastapi import FastAPI, HTTPException
import google.generativeai as genai

app = FastAPI(title="EduStack AI Engine")

@app.post("/api/rag/generate-pyg")
async def generate_pyq(payload: dict):
    prompt = payload.get("prompt")
    if not prompt:
        raise HTTPException(status_code=400, detail="Prompt is required")
    model = genai.GenerativeModel("gemini-1.5-flash")
    response = await model.generate_content_async(prompt)
    return {"success": True, "data": response.text}
```

Razorpay Payment Gateway & Cryptography

3-step payment lifecycle, HMAC-SHA256 verification, timingSafeEqual constant-time check

EduStack Codebase Specification

```
// server/services/razorpayService.js - Payment Signature Verification
const crypto = require("crypto");

const verifySignature = (orderId, paymentId, signature) => {
  const expected = crypto
    .createHmac("sha256", process.env.RAZORPAY_KEY_SECRET)
    .update(`${orderId}|${paymentId}`)
    .digest("hex");

  return crypto.timingSafeEqual(
    Buffer.from(expected, "hex"),
    Buffer.from(signature, "hex")
  );
};
```

Cloud Deployment & Production Setup

Render.com web services, MongoDB Atlas, Cloudinary zero-disk upload, env setup

-
- **Render.com Deployment:** Node.js web service running app.js with environment variables set in Render dashboard.
 - **MongoDB Atlas:** Production M0/M10 managed MongoDB cluster with IP access rules.
 - **Cloudinary CDN:** Zero-disk RAM buffer streaming via Multer memoryStorage and cloudinary.uploader.upload_stream().
 - **Environment Variables:** JWT_SECRET, MONGO_URI, RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, CLOUDINARY_URL, ML_SERVICE_URL, ADMIN_EMAILS.

Maintenance, Troubleshooting & Operational Playbook

Monitoring, log management, error recovery, and operational procedures

-
- **Morgan HTTP Logging:** `morgan("combined")` outputs Apache standard request logs for production auditing.
 - **Unhandled Exceptions:** `process.on("unhandledRejection")` logs error and gracefully shuts down HTTP server before exiting.
 - **Health Check Endpoint:** `GET /health` returns DB connection state and microservice connectivity for Render uptime checks.